

# NAPCO Nucleus

## System Behavior | Requirement Management Workflow

*What runs automatically, what Titu triggers manually, what the system produces*

### OVERVIEW

#### What this document covers

NAPCO Nucleus turns the client communications that arrive every day — Teams chat, Teams audio calls, email, and Google Drive files — into a verified requirements document and a ready-to-send client email. This document explains exactly what runs automatically, what Titu triggers manually, and what the system produces on each run.

**One operator action.** Background processes capture content into a shared store. The operator clicks **Run workflow** on GitHub when they want a verification email drafted. The agent never sends — every email leaves Gmail with an explicit click from Titu.

### CAPTURE

#### How each input channel reaches central

All four channels stage their content into a single shared store (the central share at \\172.16.205.209\nucleus-central). No operator action is required for capture — it happens continuously.

| Channel     | Mechanism                                                                                                                                               | Cadence                                                                                             | Lands at                      |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------|
| Teams chat  | Reads each dev's local Teams cache (no API, no token). Bundles recent messages + attachments the dev has downloaded.                                    | Every 15 min, per dev's PC, BD 18:00–01:00 only (once-daily 18:00 backfill catches the daytime gap) | <central>/<dev>/<date>/chat/  |
| Teams calls | Voice daemon listens for “start recording” / “Allah Hafez” phrases during a Teams call. Records mic + speaker as separate tracks. Auto-uploads on stop. | On call start/stop, per dev's PC, BD 18:00–01:00 only (gate matches the chat-push window)           | <central>/<dev>/<date>/calls/ |
| Email       | IMAP poll of khasan@ael-bd.com. Extracts body + attachment text (PDF / Word / Excel / TXT).                                                             | Every 15 min, on the agent host (MVPACCESS)                                                         | <central>/email/<date>/       |

# NAPCO Nucleus

## System Behavior | Requirement Management Workflow

*What runs automatically, what Titu triggers manually, what the system produces*

| Channel      | Mechanism                                                                                                          | Cadence                                   | Lands at                |
|--------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------|-------------------------|
| Google Drive | Service-account read of the configured folder. Downloads each new file; same extractor stack as email attachments. | Every 15 min, +5 min offset, on MVPACCESS | <central>/drive/<date>/ |

Each channel uses a dedup record (UID checkpoint for email, file-id for Drive, per-message ID for chat, per-stamp filename for calls) so re-runs never duplicate the same content. Failures on one channel never block another — if Drive is unreachable, email + chat + calls still flow.

**Dev-machine capture is on a BD evening clock.** The chat push (15-min cron) and the voice daemon recorder only fire during BD 18:00–01:00 — that's the active work window. A one-shot 18:00 chat backfill (`--last-minutes 1080`) catches any daytime messages on the next window open, so nothing is lost. Email + Drive stagers on the agent host run 24×7 and are not subject to this gate. Manual triggers (`scripts\requirement-management.bat`, `--allow-any-call` on the voice daemon) bypass the window for ad-hoc / demo use.

### TRIGGER

## The one workflow Titu runs manually

Identification of requirements + drafting of the client email is on-demand. It runs only when Titu clicks the **Requirement Management** button in the GitHub Actions tab (`napco-labs/napco-nucleus` repository). There is **no schedule**, no auto-fire, no cron for the identify step.

### Two ways to trigger the same pipeline.

- **GitHub Actions UI** — click “Run workflow” on the Requirement Management workflow, fill the client name and time window, click Run.
- **Local one-click** — double-click `scripts\requirement-management.bat` on Titu's machine. Same result.

### Inputs the operator provides

| Input        | Purpose                                                                                                                | Default           |
|--------------|------------------------------------------------------------------------------------------------------------------------|-------------------|
| client       | Substring filter on client name. Pass <code>all</code> to process every client in scope.                               | <code>all</code>  |
| last_minutes | How far back to look for chat / email / Drive content. <code>2880</code> = 48 hours, <code>60</code> = last hour, etc. | <code>2880</code> |

# NAPCO Nucleus

## System Behavior | Requirement Management Workflow

*What runs automatically, what Titu triggers manually, what the system produces*

| Input          | Purpose                                                                          | Default |
|----------------|----------------------------------------------------------------------------------|---------|
| <b>day</b>     | Date of recordings to scan (YYYY-MM-DD). Empty means today.                      | today   |
| <b>dry_run</b> | If true: collect + transcribe but skip identify + draft. Useful for diagnostics. | false   |

### ACTION

## What happens when Titu clicks Run

The workflow executes on a self-hosted runner on MVPACCESS, pulls the latest code from main, and walks through the eight steps below. Typical wall-clock: 1–3 minutes if no calls fall in the window; 15–25 minutes if a 1-hour Bangla call needs transcribing.

| Step                                            | What it does                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Walk the central share</b>                | Enumerate today's chat .docx files, call .wav + .json pairs, and chat attachments for every developer.                                                                                                                                                                                                                                                                                                |
| <b>2. Reset the unified session document</b>    | Start a clean session.docx so this run's output is independent of any prior run.                                                                                                                                                                                                                                                                                                                      |
| <b>3. Pull fresh email + Drive</b>              | Live IMAP and Drive API calls catch anything that landed in the last 15 minutes since the staging cron last ran.                                                                                                                                                                                                                                                                                      |
| <b>4. Transcribe call recordings</b>            | Whisper large-v3, chunked into 5-minute pieces and run in parallel across 4 CPU threads. Bangla audio is transcribed verbatim; Claude translates meaning to English in step 6. Low-confidence segments are flagged for the reviewer.                                                                                                                                                                  |
| <b>5. Assemble the unified session document</b> | Append each call transcript + chat + email + Drive file as a labelled section with a stable Source ID for citation.                                                                                                                                                                                                                                                                                   |
| <b>6. Identify requirements</b>                 | Claude reads the session doc, resolves the client (NAPCO Security for @napcosecurity.com, internal AEL stakeholders by individual name), pulls the client's past requirements + open in-flight items, and extracts each new requirement with: title, summary, source citations, confidence, rationale, priority (P0-P3), severity (S1-S3), audio time ranges, and conflict flags against prior items. |

# NAPCO Nucleus

## System Behavior | Requirement Management Workflow

*What runs automatically, what Titu triggers manually, what the system produces*

| Step                               | What it does                                                                                                                                                                                                                                                                                          |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7. Write the verification document | Requirements Verification <date>.docx — a numbered list, one paragraph per requirement, with citation + confidence + audio links shown below each item.                                                                                                                                               |
| 8. Draft the client email          | Builds an .eml with two attachments (the verification doc + the raw session doc), picks the right tone template based on client name (formal for NAPCO Security, informal for AEL stakeholders), pushes the draft to [Gmail]/Drafts. A prior same-subject draft is replaced so re-runs don't pile up. |

### OUTPUT

## What every run produces

| Artifact              | Audience                     | Notes                                                                                                      |
|-----------------------|------------------------------|------------------------------------------------------------------------------------------------------------|
| Verification document | Client                       | Numbered list of requirements with confidence + priority badges; attached to the verification email.       |
| Pull session document | Client (optional) + reviewer | Raw source material the requirements were drawn from. Lets the client cross-check anything that looks off. |
| Gmail draft           | Operator (Titu)              | Lives in [Gmail]/Drafts. Operator reviews + edits + sends manually. The agent never sends.                 |
| Memory update         | System (internal)            | Each requirement recorded against the client so the next run dedupes and surfaces follow-ups.              |
| Pipeline trace        | Operator (debug only)        | Per-stage record of what Claude saw + said + called. Replayable when an output is surprising.              |

### QUALITY

## How the system surfaces its own uncertainty

Speech-to-text and language-model extraction are imperfect, especially on real client audio with accent and background noise. The system is built around human-in-the-loop review — the operator always reviews the draft before sending. To make that review fast, the system explicitly marks the lines that deserve closer attention:

- **Uncertain audio segments** — transcript lines where Whisper's per-segment confidence was low get an (*uncertain*) tag so the reviewer knows where to re-listen.

# NAPCO Nucleus

## System Behavior | Requirement Management Workflow

*What runs automatically, what Titu triggers manually, what the system produces*

- **Low-confidence requirements** — items the model rates below 0.75 are shown in amber with a (review) marker in the verification document.
- **Conflict warnings** — if today's content contradicts an open item from a prior session, a ■ **Possible conflict** line is shown.
- **Audio anchors** — for any requirement extracted from a call, the document shows the exact HH:MM:SS-HH:MM:SS range so the reviewer can pull a 30-second snippet for spot-check.
- **Calibration feedback** — the system tracks how often the reviewer keeps vs. rejects items by confidence band, and feeds that signal into the next run's prompt so the model's stated confidence becomes more accurate over time.

### OPERATOR

## Titu's daily role

| Action                                                               | Frequency                                                       |
|----------------------------------------------------------------------|-----------------------------------------------------------------|
| Keep Teams desktop signed in                                         | Continuously (background only)                                  |
| Voice daemon running on laptop                                       | Continuously; active recording only fires during BD 18:00–01:00 |
| Say a phrase like “start recording” / “Allah Hafez” on Teams calls   | Per client call (a few seconds of effort)                       |
| Click “Run workflow” on Requirement Management in GitHub Actions     | Whenever a verification email is needed                         |
| Review the draft email in [Gmail]/Drafts, edit if needed, click Send | Once per run (typically 3–10 minutes per session)               |

All other behaviour — capture, transcription, requirement extraction, conflict detection, memory of past sessions — happens automatically on the agent host. The team's daily work is unaffected: developers carry on having client calls and exchanging chat messages exactly as before. The system observes, organizes, and produces a draft. Titu approves and sends.

### RELIABILITY

## How the system handles failure

- **Retry with backoff** on transient IMAP / Drive / SMTP failures — a single network blip never costs a run.
- **Per-stage timeouts** — a stuck stage aborts cleanly instead of blocking indefinitely.

# NAPCO Nucleus

## System Behavior | Requirement Management Workflow

*What runs automatically, what Titu triggers manually, what the system produces*

- **Cross-process locking** — if Titu clicks Run Workflow twice within seconds, the second invocation waits for the first to finish instead of corrupting shared state.
- **Idempotent draft replacement** — re-running for the same day replaces the prior Gmail draft instead of stacking duplicates.
- **Per-run trace artifact** — if a run fails, the raw session document is uploaded for debugging.

**Bottom line.** The system captures every channel automatically, requires one click from Titu when a draft is needed, and produces a verification email ready for his review. It is conservative about its own accuracy — uncertainty is surfaced, never hidden, and no email leaves Gmail without Titu's explicit action.